

ATIVIDADES

Objetivo

Consolidar os conceitos fundamentais de lógica de programação em **JavaScript**. O foco principal será a aplicação prática de estruturas **condicionais**, **laços de repetição** e a **manipulação de fluxos via terminal**, explorando o uso de **funções nativas** da linguagem para a resolução de problemas reais.

Atividade 1

Desenvolva um programa que simule o lançamento de um dado. O programa deve perguntar ao usuário qual é o número de lados do dado, e em seguida, simular um lançamento com um resultado aleatório, exibindo o resultado.

Cuidado: não se esqueça de incluir o 1.

Atividade 2

Crie um programa que permita ao usuário jogar **Pedra, Papel e Tesoura** contra o computador. O programa deve sortear a escolha do computador e determinar o vencedor.

Ideia:

1. Definir um array `opcoes` com Pedra, Papel e Tesoura dentro.
2. Inicializar `escolhaComputador`, `escolhaUsuario` e `resultado` como strings vazias.
3. Usar o `process.stdin` para pegar a `escolhaUsuario`.
4. Definir `escolhaComputador` como uma posição aleatória do vetor `opcoes`.
5. Implementar a lógica:
 - a. Se `escolhaUsuario == escolhaComputador`, `resultado = Empate`.
 - b. Senão se `escolhaUsuario == 'Pedra' && escolhaComputador == 'Tesoura' || ... resultado = 'Você venceu!'`.
 - c. Senão `resultado = 'Você perdeu!'`.
6. Finalmente, mostrar usando `console.log()` a escolha do computador, do usuário e o resultado.

Desafio:

Implemente a expansão do jogo Pedra, Papel e Tesoura que é conhecida como **Pedra, Papel, Tesoura, Lagarto e Spock**.

As regras de Pedra-papel-tesoura-lagarto-Spock são:

- Tesoura corta papel
- Papel cobre pedra
- Pedra esmaga lagarto
- Lagarto envenena Spock
- Spock esmaga (ou derrete) tesoura
- Tesoura decapita lagarto
- Lagarto come papel
- Papel refuta Spock
- Spock vaporiza pedra
- Pedra amassa tesoura

Referência: <https://pt.wikipedia.org/wiki/Pedra-papel-tesoura-lagarto-Spock>

Atividade 3

Crie um programa que gere uma senha aleatória de um comprimento especificado pelo usuário. A senha deve conter letras maiúsculas, minúsculas e números.

Ideia:

1. Definir uma string (eu chamei de `caracteres`) que contém todos os caracteres possíveis que podem compor a senha. Esta string inclui letras maiúsculas, minúsculas e números. Não precisa ser Array.
2. Inicialize a variável `senha` como uma string vazia. Esta variável armazenará a senha gerada mais tarde.
3. Utilizar o `process.stdin` para perguntar ao usuário o comprimento da senha desejada. A resposta do usuário é armazenada na variável `comprimentoSenha` que é então convertida para um número inteiro usando `parseInt()`.
4. Usar um `for`(let `i = 0`; `i < comprimentoSenha`; `i++`) para gerar cada caractere da senha.
 - a. Usamos `Math.random()` para gerar um número aleatório entre 0 e 1. Multiplicamos este número pelo comprimento da string `caracteres` e aplicamos `Math.floor()` para obter um índice inteiro aleatório dentro do intervalo de índices da string.
 - b. Com esse número usamos `caracteres.charAt(numeroAleatorio)` para obter o caractere da string `caracteres` no índice aleatório gerado e o concatenamos à variável `senha`.

5. Depois do laço, só exibir a senha.

```
const caracteres = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789';
```

Atividade 4

Crie um programa que sorteie um número entre 1 e 100. O usuário deve tentar adivinhar o número sorteado. O programa deve fornecer dicas se o palpite estiver acima ou abaixo do número sorteado até que o usuário adivinhe corretamente.

1. Você irá precisar do `readline`. Lembre-se de importar e criar a interface com input e output.
2. Para sortear um número você precisará utilizar `Math.random()` e `Math.floor()`. Fazer uma função que irá sortear e retornar o número sorteado.
3. Crie uma função para solicitar o palpite ao usuário, desta forma não precisa utilizar laço de repetição e dentro dela fazer a seguintes verificação:
 - a. Se o palpite for menor que o número sorteado, exibe “Muito Abaixo!” e solicita outro palpite.
 - b. Se o palpite for maior que o número sorteado, exibe “Muito Alto!” e solicita outro palpite.
 - c. Se o palpite estiver correto, exibe “Parabéns!” e fecha a interface `readline`.

Desafio:

1. Implemente mensagem de boas vindas ao iniciar o programa
2. Informe ao jogador quantas rodadas foram necessárias para ele acertar
3. Limite máximo de rodadas
4. Avise quando o número que o usuário digitou for inválido
5. Usar `switch` para tratar casos maior, menor e acertou.